

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	G. Santosh Kumar		Reg. No.: 192724020
Section / Batch:	2025 Batch		Date: 26/08/2026

Code of Conduct

I, G. Santosh Kumar (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: G. Santosh Kumar

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)

- Clustering effect
- Memory usage
- Performance under high load factor
- Which method is more suitable for large-scale applications and why?

2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)

3. Compare Quadratic Probing and Double Hashing in terms of:

- Clustering issues
- Collision resolution efficiency
- Suitability for dense hash tables

4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)

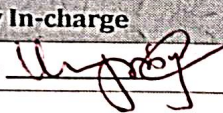
- Practical significance
- Impact on algorithm selection

5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)

- Space complexity (stack usage)
- Ease of implementation
- Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

I. Linear probing vs Separate chaining

fact	Linear probing	Separate chaining
collision handling	searches next available slot	stores collided keys in a linked list (bucket)
clustering	high primary clustering	no primary clustering
memory usage	lower; uses table itself	higher; requires extra words
cache performance	usually very good	generally poorer due to pointer traversal.
load factor	performance decreases rapidly when high	can work reasonably even when load factor > 1
Average search	$O(1)$ when table is lightly loaded	$O(1)$ with a good hash function
worst-case search	$O(n)$	$O(n)$
Insertion	$O(1)$ average	$O(1)$ average
Deletion	more complicated	Relatively easy
Implementation	Simple	Requires linked-list

(a) clustering effect

Linear probing suffers from primary clustering. when several consecutive positions become occupied, new collisions cause longer consecutive groups.

Ex:-

hash table:

[10] [20] [30] [] [] []

If another key maps to position 1:

[10] [20] [30] [40] [] []

(b) memory usage

linear probing is memory efficient because all elements are stored directly inside the hash table. quick

Separate chaining requires additional memory for linked-list nodes (or) bucket structures.

Therefore -

linear probing $\rightarrow$ less extra memory

(c) performance under high load factor

load factor :

$$\alpha = \frac{n}{m}$$

linear probing becomes inefficient as α approaches 1 because finding an empty position requires many probes.

which is suitable for large-scale applications.

Separate chaining is generally more suitable for large scale applications, especially when the no. of elements is unpredictable.

Quick sort vs merge sort for a linked list

Factor	Quick sort	merge sort
Avg time	$O(n \log n)$	$O(n \log n)$
worst-case time	$O(n^2)$	$O(n \log n)$
Best case	$O(n \log n)$	$O(n \log n)$
Extra space of linked list	$O(\log n)$ average recursion	$O(n \log n)$ recursion
Stability	usually not stable	stable
linked-list suitability	moderate	excellent
Sequential access	less natural	excellent
Cache behaviour	depends on partitioning	sequential traversal
worst-case guarantee	no	yes

Quick sort on linked list

Quick sort works by:

1. selecting a pivot
2. partitioning elements around the pivot
3. Recursively sorting the partition.

$$T(n) = 2T(n/2) + O(n)$$

Therefore: $T(n) = O(n \log n)$

in an iterative implementation, putting both subarrays
indly can consume unnecessary memory. A common
optimization is to process the smaller partition first
& keep the larger partition for later.

Ugong